

Node.js 15-Minute Daily Recap Cheatsheet (Day 1 - Day 6)

DAY 1 — ARCHITECTURE & CORE MODULES

V8 Engine, Call Stack & Heap

V8 JIT-compiles JS directly to machine code; the Call Stack executes synchronous function frames (LIFO), while the Heap stores dynamic objects and closures. Consistent object shapes prevent de-optimizations; stack overflows occur when the call limit (~10k frames) is exceeded.

```
function greet(name) { return `Hi, ${name}`; } // pushed to Call Stack,
executed, popped
const user = { name: "Dev" }; // pointer on Call Stack, allocated in
dynamic Heap memory
```

libuv & Thread Pool

libuv provides non-blocking async I/O via OS primitives (epoll/kqueue) and maintains a 4-thread pool for blocking tasks (fs, crypto, zlib, dns.lookup). Network I/O uses OS sockets directly without consuming thread pool workers.

```
process.env.UV_THREADPOOL_SIZE = '8'; // expand thread capacity before
async calls
require('crypto').pbkdf2('pass', 'salt', 100000, 64, 'sha512', () => {});
// runs in pool
```

Event Loop Phases

Cycles through 6 phases: Timers -> Pending -> Idle/Prepare -> Poll -> Check -> Close. Between EVERY phase transition, microtasks drain completely (`process.nextTick` queue drains before `Promise.then`).

```
setTimeout(() => console.log('Timers'), 0); // scheduled in Timers phase
setImmediate(() => console.log('Check')); // scheduled in Check phase
```

process.nextTick() vs setImmediate()

`process.nextTick` runs immediately after the current operation before advancing to any loop phase (recursive calls starve I/O). `setImmediate` queues specifically in the Check phase, safely yielding to pending I/O.

```
process.nextTick(() => console.log('Microtask: runs first')); // can starve I/O if recursive
setImmediate(() => console.log('Check phase: yields to I/O')); // safe for recursive tasks
```

setTimeout(fn, 0)

Schedules a callback in the Timers phase after a minimum OS delay (clamped to 1ms in Node.js). Inside an I/O callback, `setImmediate` is guaranteed to execute before `setTimeout(fn, 0)`.

```
require('fs').readFile(__filename, () => {
  setTimeout(() => console.log('Timers next loop'), 0); // executes second
  setImmediate(() => console.log('Check current loop')); // executes first
  guaranteed
});
```

Worker Threads

Runs JavaScript threads in parallel within the same process to handle CPU-intensive tasks without blocking the main event loop. Workers run separate V8 isolates and communicate via message channels or `SharedArrayBuffer`.

```
const { Worker, isMainThread, parentPort } = require('worker_threads');
if (isMainThread) new Worker(__filename).on('message', console.log); // main thread
else parentPort.postMessage(42); // worker execution thread
```

Cluster Module

Forks multiple independent OS processes that share the same network port to load-balance across CPU cores. Each process has its own memory, event loop, and V8 isolate; state must be centralized externally in stores like Redis.

```
const cluster = require('cluster'), http = require('http'), os = require('os');
if (cluster.isPrimary) os.cpus().forEach(() => cluster.fork()); // fork process per core
else http.createServer((req, res) => res.end('OK')).listen(3000); // shared server port
```

CommonJS (CJS)

Synchronous module system where required files are evaluated at runtime and cached as singletons. Never reassign `exports = {}` directly, as doing so breaks the reference to `module.exports`.

```
module.exports = { add: (a, b) => a + b }; // correctly assigns exports
const { add } = require('./math');        // evaluates synchronously and
                                           caches instance
```

require() Resolution Algorithm

Checks core modules first, then probes explicit relative/absolute paths (`.js`, `.json`, `.node`, directory `index.js`). If not a path, it walks up `node_modules` until reaching the system root.

```
console.log(require.resolve('express')); // inspect exact resolved path in
node_modules
delete require.cache[require.resolve('./config')]; // evict cache to force
reload
```

ECMAScript Modules (ESM)

Standardized static module format parsed and resolved before runtime, enabling dead-code elimination (tree-shaking) and top-level await. Requires `"type": "module"` in `package.json`; replace `__dirname` with `import.meta.url`.

```
export const pi = 3.14; // named export
import { fileURLToPath } from 'url'; const __dirname =
fileURLToPath(import.meta.url);
```

Circular Dependencies

Occurs when module A imports B and B imports A; Node resolves the deadlock by returning a partially-evaluated `module.exports` object. Resolve by extracting shared dependencies into a third module or using lazy inside-function imports.

```
// a.js: const b = require('./b'); module.exports = { name: 'A' };
// b.js: const a = require('./a'); // `a` is an empty object {} here
because a.js is still loading!
```

fs Module

Provides file system operations; always use `fs/promises` with `async/await` in web servers. Never run blocking sync methods like `readFileSync` in route handlers as they freeze the entire server.

```
const fs = require('fs/promises');
const data = await fs.readFile('./config.json', 'utf8'); // non-blocking
async file read
```

path Module

Utilities for normalizing and constructing paths across operating systems (Windows \ vs POSIX /).
Combine `__dirname` with `path.join()` and sanitize file upload names using `path.basename()`.

```
const path = require('path');
const file = path.join(__dirname, 'uploads',
path.basename('../evil/doc.pdf')); // strips traversal
```

Streams & Backpressure

Streams process data chunk-by-chunk (~64KB) to maintain low memory overhead during large file or network transfers. Backpressure occurs when `writable.write()` returns `false`, signaling the readable stream to pause until the `drain` event fires.

```
const { pipeline } = require('stream/promises'); // handles cleanup &
errors automatically
await pipeline(fs.createReadStream('in.txt'), zlib.createGzip(),
fs.createWriteStream('out.gz'));
```

EventEmitter

Observer/Pub-Sub implementation powering HTTP servers, Streams, and Sockets across Node.js core. Always register an `.on('error')` handler; emitting an unhandled error event will crash the Node.js process immediately.

```
const emitter = new (require('events'))();
emitter.on('error', (err) => console.error(err)); // prevents unhandled
crash
emitter.emit('event', { id: 1 }); // synchronously invokes
listeners
```

crypto Module

Wraps OpenSSL for cryptographic hashing, HMAC signing, cipher streams, and random byte generation. Always verify secrets and signatures using `crypto.timingSafeEqual` to avoid side-channel timing attacks.

```
const crypto = require('crypto');
const token = crypto.randomBytes(32).toString('hex'); // cryptographically
secure token
const match = crypto.timingSafeEqual(Buffer.from(sig1), Buffer.from(sig2));
// constant time
```

process Object

Global runtime object providing environment variables, process metrics, and OS signal handling. Listen for **SIGTERM** and **SIGINT** signals to gracefully close active HTTP connections and database pools.

```
process.on('SIGTERM', async () => {
  server.close(); await db.disconnect(); process.exit(0); // clean
  container shutdown
});
```

Buffer

Fixed-length sequences of raw binary bytes allocated outside V8's garbage-collected heap. Used for binary streaming, image manipulation, and encoding conversions (utf-8, base64, hex); avoid **Buffer.allocUnsafe()**.

```
const buf = Buffer.from('Node.js', 'utf8'); // creates raw binary buffer
const base64 = buf.toString('base64');      // encodes raw bytes into
base64 string
```

DAY 2 — HTTP, EXPRESS & ASYNC FLOWS

Request & Response Anatomy

Requests contain a Request Line (Method, URL), metadata Headers, and an optional Body; responses return a Status Line, Headers, and Body. Always use **return res.status(...)** to prevent runtime "Cannot set headers after they are sent" crashes.

```
app.post('/api/users', (req, res) => {
  if (!req.body.name) return res.status(400).json({ error: 'Missing name'
}); // return stops execution
  res.status(201).json({ created: true });
});
```

Status Codes

Establish clear API contracts: 2xx (Success), 4xx (Client mistakes), 5xx (Server crashes). 401 = Unauthenticated ("Who are you?"), 403 = Forbidden ("I know who you are, but you cannot do this"), 400 = Malformed syntax, 422 = Validation failure.

```
res.status(401).json({ error: 'Unauthenticated' }); // missing/invalid token
res.status(403).json({ error: 'Forbidden' }); // authenticated user lacks permissions
```

CORS (Cross-Origin Resource Sharing)

Browser security policy restricting scripts on one domain from reading resources from another origin. Non-simple requests (PUT, DELETE, custom auth headers) send an automatic preflight **OPTIONS** call before the main request.

```
const cors = require('cors');
app.use(cors({ origin: 'https://client.com', credentials: true })); // allow origin + cookies
```

Cookies: HttpOnly, Secure, SameSite

Server-issued state stored in the browser and attached automatically to future matching requests. Prevent token theft by setting **httpOnly: true** (unreadable to JS), **secure: true** (HTTPS), and **sameSite: 'strict' | 'lax'** (mitigates CSRF).

```
res.cookie('token', jwt, { httpOnly: true, secure: true, sameSite: 'strict' }); // protected auth cookie
```

Express Under the Hood & Middleware

Express is a thin routing and middleware wrapper built directly over Node's native **http.createServer()**. Middleware functions execute sequentially with access to (**req**, **res**, **next**); you must call **next()** or send a response to avoid hanging requests.

```
const http = require('http'), express = require('express'), app = express();
app.use((req, res, next) => { req.startTime = Date.now(); next(); }); // attaches custom property
http.createServer(app).listen(3000); // Express mounted on native Node server
```

Error Middleware (4 Arguments)

Express detects error-handling middleware exclusively by checking for 4 parameters: (`err`, `req`, `res`, `next`). Register this handler at the bottom of the middleware stack to capture errors routed via `next(err)`.

```
app.use((err, req, res, next) => {  
  res.status(err.statusCode || 500).json({ error: err.message }); //  
  centralized error response  
});
```

Request Parsing & `express.Router()`

Express parses inputs across `req.params` (URL paths), `req.query` (query parameters), and `req.body` (JSON payload). Use `express.Router({ mergeParams: true })` when nesting sub-routes to retain access to parent parameters.

```
const router = require('express').Router({ mergeParams: true });  
router.get('/posts/:postId', (req, res) => res.json(req.params)); //  
captures parent + child params
```

Promise Combinators

Utility functions to coordinate concurrent promises based on failure tolerance.

```
await Promise.all([p1, p2]); // resolves when all succeed; fails  
fast if any reject  
await Promise.allSettled([p1, p2]); // resolves when all complete; returns  
status & values  
await Promise.race([p1, timeout]); // returns the first settled promise  
(used for timeouts)  
await Promise.any([p1, p2]); // returns first resolved promise;  
rejects only if all fail
```

for await...of

Iterates sequentially over async iterables without loading the entire collection into memory at once. Ideal for reading paginated database cursors and chunked file streams.

```
for await (const chunk of fs.createReadStream('big.txt')) {  
  processChunk(chunk); // streams items one chunk at a time  
}
```

The Async Array Trap (`forEach`)

`Array.prototype.forEach` executes synchronously and ignores returned promises, failing to await asynchronous callbacks. Use `for...of` loops for sequential operations, or `Promise.all(arr.map())` for concurrent execution.

```
// BAD: items.forEach(async (i) => await save(i)); // runs un-awaited!  
for (const item of items) { await save(item); } // GOOD: waits for  
completion sequentially
```

Operational vs Programmer Errors

Operational errors are expected runtime conditions (bad inputs, expired tokens, DB timeouts) handled gracefully with responses. Programmer errors are bugs (null pointer reads, syntax errors) requiring process termination and restart.

```
if (!user) throw new NotFoundError('User not found'); // operational error  
(recoverable)  
// TypeError: user.name.toUpperCase() when user is null -> programmer error  
(crash/restart)
```

Custom Error Classes & asyncWrapper

Custom `AppError` classes standardize operational error metadata (`statusCode`, `isOperational`). The `asyncWrapper` utility catches unhandled promise rejections in controllers and routes them directly to `next(err)`.

```
class AppError extends Error { constructor(msg, code) { super(msg);  
this.statusCode = code; } }  
const asyncWrapper = (fn) => (req, res, next) => Promise.resolve(fn(req,  
res, next)).catch(next);
```

uncaughtException & unhandledRejection

Global process safety nets for uncaught synchronous bugs and unhandled rejected promises. Log the stack trace and exit via `process.exit(1)` so orchestrators (Docker, PM2, K8s) can restart an uncorrupted process.

```
process.on('unhandledRejection', (reason) => {  
  console.error('Unhandled:', reason); server.close(() => process.exit(1));  
  // clean exit  
});
```

DAY 3 — TYPESCRIPT, ZOD, POSTGRESQL & PRISMA

tsconfig.json Essentials

Strict configuration enforces compile-time safety: `"strict": true` enables strict checks like `strictNullChecks`. Set `"noUncheckedIndexedAccess": true` to treat array lookups (`arr[0]`) as `T | undefined` to prevent undefined property crashes.

```
{
  "compilerOptions": { "strict": true, "noUncheckedIndexedAccess": true,
    "outDir": "./dist" }
}
```

Declaration Merging

TypeScript merges multiple declarations of the same interface into a single composite type definition. Used to extend Express's global `Request` interface with custom fields like `req.user` without unsafe `as any` casts.

```
declare global {
  namespace Express { interface Request { user?: { id: string; role: string
}; } }
} // TypeScript now types req.user safely across all routes
```

Generics & Generic Repository

Parameterized types enable reusable abstractions while maintaining complete compile-time type safety. Used to construct generic database repositories that enforce CRUD contracts across entity models.

```
interface Repository<T> { findById(id: string): Promise<T | null |>; }
class UserRepo implements Repository<User> { async findById(id: string) {
return null; } }
```

Utility Types (Partial, Pick, Omit, ReturnType, Awaited)

Derive new shapes from existing interfaces without code duplication: `Partial` makes all fields optional (PATCH updates), `Omit` strips properties (hiding passwords), and `Awaited<ReturnType>` extracts return types from async methods.

```
type UserUpdate = Partial<User>; // all fields optional for
PATCH endpoints
type PublicUser = Omit<User, 'passwordHash'>; // removes sensitive fields
from API responses
type UserPromise = Awaited<ReturnType<typeof getUser>>; // unwraps async
function return type
```

Discriminated Unions & Type Guards

Discriminated unions combine types using a shared literal tag for clean type narrowing. Custom type guards (`val is AppError`) inspect runtime properties to help TypeScript narrow broad types.

```
type Result<T> = { ok: true; data: T } | { ok: false; error: string };
function isError(val: any): val is Error { return val instanceof Error; }
// narrows type safely
```

Zod Schemas & Inferred Types

Zod defines runtime validation schemas for untrusted client inputs and infers TypeScript types automatically. `z.infer<typeof schema>` extracts static types directly from validation schemas, keeping them in sync.

```
import { z } from 'zod';
const UserSchema = z.object({ email: z.string().email(), age:
z.number().min(18) });
type UserDto = z.infer<typeof UserSchema>; // derived TypeScript interface
```

.parse() vs .safeParse()

`.parse()` validates incoming data and throws a `ZodError` immediately upon failure. `.safeParse()` captures errors internally and returns a clean `{ success: true, data }` or `{ success: false, error }` result object.

```
const result = UserSchema.safeParse(req.body);
if (!result.success) return res.status(422).json({ errors:
result.error.errors }); // clean control flow
```

Zod Express Validation Middleware

Reusable middleware factory validating incoming `body`, `query`, and `params` against Zod schemas. Reassigns parsed and transformed data directly to `req.body`, ensuring downstream handlers receive sanitized input.

```
const validate = (schema: z.AnyZodObject) => async (req, res, next) => {
  const parsed = await schema.safeParseAsync({ body: req.body });
  if (!parsed.success) return res.status(422).json(parsed.error);
  req.body = parsed.data.body; next(); // reassigns sanitized, typed data
};
```

SQL Core Commands & Joins

INNER JOIN returns rows with matches in both tables; **LEFT JOIN** returns all left rows, filling missing right columns with **NULL**. Always specify explicit columns in **SELECT** queries rather than using **SELECT *** in production.

```
SELECT users.name, posts.title FROM users
LEFT JOIN posts ON posts.user_id = users.id WHERE users.deleted_at IS NULL;
```

Database Relationships (1:1, 1:N, M:N)

1:1 maps one row to another using a **UNIQUE** foreign key; 1:N stores the foreign key on the child ("many") table. M:N connects records via a dedicated junction table holding composite foreign keys from both related tables.

```
CREATE TABLE post_tags (
  post_id UUID REFERENCES posts(id) ON DELETE CASCADE,
  tag_id UUID REFERENCES tags(id) ON DELETE CASCADE,
  PRIMARY KEY (post_id, tag_id) -- composite PK enforces M:N integrity
);
```

Database Indexes & ACID Transactions

B-tree indexes accelerate **WHERE** filters and **JOIN** queries by avoiding expensive full-table scans. ACID transactions wrap multiple SQL mutations inside **BEGIN/COMMIT/ROLLBACK** blocks to guarantee atomicity.

```
CREATE INDEX idx_users_email ON users(email); -- speeds up lookups
BEGIN; UPDATE accounts SET bal = bal - 50 WHERE id = 1; UPDATE accounts SET
bal = bal + 50 WHERE id = 2; COMMIT;
```

N+1 Query Problem

Occurs when fetching 1 parent collection, followed by N sequential queries to resolve child relations. Fix by running single SQL **JOIN** statements, or using Prisma's **include** and DataLoader patterns to batch calls.

```
// BAD: const users = await db.users(); for (u of users) await
db.posts(u.id); // 101 queries
const users = await prisma.user.findMany({ include: { posts: true } }); //
1 batched query
```

Connection Pooling

Maintains a warm pool of reusable database connections, avoiding the 20-50ms TCP/TLS handshake latency on every request. Create your pool once at startup; never instantiate connection pools inside request handlers.

```
import { Pool } from 'pg';
const pool = new Pool({ max: 20, idleTimeoutMillis: 30000 }); // reusable
connection pool
const res = await pool.query('SELECT NOW()'); // grabs connection from
pool, returns when done
```

Prisma ORM Migrations & Queries

Declarative database management: use `prisma migrate dev` locally to generate migration SQL files. In CI/CD and production environments, use `prisma migrate deploy` to safely apply pending migrations without prompts.

```
const user = await prisma.user.upsert({
  where: { email: 'a@b.com' },
  update: { name: 'Dev' },
  create: { email: 'a@b.com', name: 'Dev' } // updates or creates
atomically
});
```

Prisma: select vs include & \$transaction

`include` pulls related models with all their fields; `select` specifies exact attributes, preventing accidental exposure of sensitive columns like password hashes. Use `prisma.$transaction` with a callback to run multi-step dependent queries atomically with auto-rollback.

```
await prisma.$transaction(async (tx) => {
  await tx.wallet.decrement({ where: { id: 1 }, data: { amount: 10 } });
  await tx.wallet.increment({ where: { id: 2 }, data: { amount: 10 } }); //
atomic rollback
});
```

DAY 4 — AUTHENTICATION, AUTHORIZATION & DEBUGGING

bcrypt Hashing & Timing Attacks

bcrypt uses a slow work factor (2^{cost} iterations) and auto-embeds unique salts to defeat rainbow table lookups. Always run `bcrypt.compare` even when a user look up fails to maintain constant processing time and thwart email enumeration via timing attacks.

```
const hash = await bcrypt.hash('password123', 12); // embeds salt + cost factor in output
const valid = await bcrypt.compare(inputPass, hash); // extracts salt internally
```

JWT Anatomy (Header, Payload, Signature)

Composed of 3 base64url-encoded parts: Header (algorithm), Payload (claims like sub/exp), and an HMAC/RSA Signature. JWTs are signed to guarantee authenticity, but not encrypted; never store passwords or sensitive data in payloads.

```
const token = jwt.sign({ sub: user.id, role: 'ADMIN' }, secret, { expiresIn: '15m' });
const decoded = jwt.verify(token, secret, { algorithms: ['HS256'] }); // verifies signature
```

HS256 vs RS256

HS256 uses a single shared symmetric key to sign and verify tokens (best for simple monolithic applications). RS256 uses asymmetric cryptography: auth servers sign with a private key, while other services verify with a public key.

```
// RS256 verification: microservices only need the public key
const payload = jwt.verify(token, publicKey, { algorithms: ['RS256'] });
```

Access Token + Refresh Token Rotation

Keep access tokens short-lived (15 minutes) in memory; store long-lived refresh tokens in secure **httpOnly** cookies. Invalidate refresh tokens upon use and issue a new pair; if an invalidated token is reused, revoke all user sessions.

```
// Inside /auth/refresh:
await prisma.refreshToken.delete({ where: { tokenHash } }); // invalidate old token
const newRefresh = signRefreshToken(user.id); // issue rotated token
```

JWT Error Handling

Differentiate incoming authentication failures explicitly: **TokenExpiredError** instructs the frontend to request a refresh via **/auth/refresh**. **JsonWebTokenError** flags payload tampering, bad signatures, or structural malformation.

```
try { jwt.verify(token, secret); } catch (err) {  
  if (err instanceof jwt.TokenExpiredError) return next(new  
  AppError('Expired', 401));  
  return next(new AppError('Invalid token', 401));  
}
```

RBAC (Role-Based Access Control)

Decouples authorization logic by assigning permissions to roles, and roles to users. Middleware verifies whether the authenticated user's assigned role contains the permission needed for the endpoint.

```
const PERMISSIONS = { ADMIN: ['write:jobs', 'delete:jobs'], USER:  
  ['read:jobs'] };  
const check = (perm) => (req, res, next) =>  
  PERMISSIONS[req.user.role]?.includes(perm) ? next() : res.sendStatus(403);
```

Resource Ownership Checks (IDOR Prevention)

Insecure Direct Object References occur when authorization stops at the role level and fails to verify resource ownership. Always scope data queries directly to the authenticated caller's identity (`where: { id: req.params.id, userId: req.user.id }`).

```
const updated = await prisma.post.updateMany({  
  where: { id: req.params.id, userId: req.user.id }, data: req.body //  
  enforces ownership  
});  
if (!updated.count) throw new NotFoundError('Post'); // return 404 to avoid  
enumeration
```

Debugging Tools: VS Code & Breakpoints

Debug applications by starting Node with `--inspect` or configuring `.vscode/launch.json`. Line breakpoints pause execution, conditional breakpoints halt only when target expressions evaluate to true, and logpoints stream metrics without altering code.

```
{ "type": "node", "request": "launch", "runtimeArgs": ["-r", "ts-  
node/register"], "args": ["src/index.ts"] }
```

Missing Await Bug Detection

Calling an asynchronous function without `await` stores a pending `Promise` object rather than its resolved value. Strict TypeScript settings and the ESLint rule `@typescript-eslint/no-floating-promises` flag non-awaited calls at build time.

```
const user = getUser(id); // missing await returns Promise<User>, which is
always truthy!
// fix: const user = await getUser(id);
```

Enterprise Code Reading Order

When onboarding or triaging an issue, trace code in this sequence: Routes -> Controller -> Service -> Repository -> Database Schema. Inspect `schema.prisma` first to understand the underlying relationships, constraints, and models within minutes.

DAY 5 — ENTERPRISE ARCHITECTURE, LOGGING & SECURITY

The 4-Layer Architecture

Routes (bind paths/middleware) -> Controllers (parse HTTP/format responses) -> Services (business logic) -> Repositories (database queries). Never pass Express `req/res` into services; never make direct Prisma calls inside controllers.

```
// Controller delegates plain data; never passes `req` or `res` to the
service:
const job = await this.jobService.create(req.body, req.user.id);
res.status(201).json(job);
```

Repository Pattern & Dependency Injection

Abstract the database behind an interface to decouple the service from the underlying ORM, making services simple to unit test with in-memory mocks. Inversion of Control passes dependencies via class constructors rather than instantiating them internally.

```
class JobService {
  constructor(private jobRepo: IJobRepository) {} // inject interface
  abstraction
}
const service = new JobService(new PrismaJobRepository()); // inject real
instance
```

Code Smells: Fat Controllers & Fat Services

Controllers making direct database calls ("Fat Controllers") break modularity and prevent unit testing. Services accepting (`req`, `res`) Express objects ("Fat Services") bleed transport concerns into business logic.

```
// BAD (Fat Controller): app.post('/jobs', (req, res) =>
prisma.job.create({ data: req.body }));
// BAD (Fat Service): async create(req: Request) { const ip = req.ip; } //
coupled to HTTP!
```

Structured Logging with Pino

`console.log` produces unstructured, non-indexed string blobs that block the Node.js event loop under heavy loads. Pino outputs structured, queryable JSON logs asynchronously, classifying traces by log level (trace: 10, debug: 20, info: 30, warn: 40, error: 50, fatal: 60).

```
import pino from 'pino';
const logger = pino({ level: 'info', redact: ['password', 'token'] }); //
redacts secrets
logger.info({ userId: '42', event: 'LOGIN_SUCCESS' }, 'User logged in'); //
structured output
```

Correlation IDs (Request Tracing)

A unique UUID assigned to each incoming request via middleware and tagged across all downstream child logs. When errors trigger, filter logs by correlation ID to trace the execution history across services.

```
app.use((req, res, next) => {
  req.id = req.headers['x-correlation-id'] || crypto.randomUUID();
  req.log = logger.child({ correlationId: req.id }); next(); // attached to
  child logs
});
```

Helmet: HTTP Security Headers

Secures Express apps by setting defensive HTTP response headers automatically. Configures Content-Security-Policy (XSS), HSTS (forces HTTPS), and X-Frame-Options (prevents clickjacking within iframes).

```
import helmet from 'helmet';
app.use(helmet()); // sets HSTS, CSP, X-Frame-Options, X-Content-Type-
Options: nosniff
```

Distributed Rate Limiting with Redis

In-memory rate limit counters fail in multi-instance or clustered deployments because each process tracks visits in isolation. Use `rate-limit-redis` to maintain a single, synchronized request count across all running instances.


```
import rateLimit from 'express-rate-limit'; import RedisStore from 'rate-limit-redis';
const limiter = rateLimit({ store: new RedisStore({ sendCommand: (...args) => redis.sendCommand(args) }), max: 100 });
```

Multer Storage: diskStorage vs memoryStorage

diskStorage streams uploads directly to the local disk, keeping memory overhead low.

memoryStorage keeps files in RAM as a **Buffer**, making it ideal for fast virus scanning, resizing, or direct uploads to cloud buckets (S3).

```
const upload = multer({ storage: multer.memoryStorage(), limits: {
  fileSize: 5 * 1024 * 1024 } }); // 5MB
```

File Type Validation via Magic Bytes

Client-provided extensions (**.png**) and **Content-Type** headers are easily spoofed by malicious actors. Inspect the buffer's initial binary signature ("magic bytes") using libraries like **file-type** to verify the actual file format.

```
import { fileTypeFromBuffer } from 'file-type';
const type = await fileTypeFromBuffer(req.file.buffer); // inspects actual binary header
if (!type || type.mime !== 'image/png') throw new ValidationError('Invalid file payload');
```

DAY 6 — TESTING, DOCKER & CI/CD

Unit Tests vs Integration Tests

Unit tests isolate a single function or class with all external dependencies mocked (runs in milliseconds). Integration tests execute real HTTP calls using **supertest** against a running test database to verify the entire system end-to-end.

```
// Unit: test business logic using mocked dependencies
// Integration: request(app).post('/users').send(payload).expect(201); // hits test DB
```

AAA Pattern (Arrange, Act, Assert)

Structure every test case into three clean, readable phases:

```
it('returns user by id', async () => {
  mockRepo.find.mockResolvedValue({ id: '1' }); // ARRANGE: set up state &
  mocks
  const user = await service.getUser('1');      // ACT: execute target
  functionality
  expect(user.id).toBe('1');                    // ASSERT: verify expected
  behavior
});
```

Mocking: `jest.fn()`, `jest.spyOn()`, `jest.mock()`

`jest.fn()` creates an isolated mock function with configurable return values. `jest.spyOn()` wraps existing methods to track calls or replace behavior; `jest.mock()` auto-mocks entire modules.

```
const fn = jest.fn().mockResolvedValue(true); // standalone mock
function
jest.spyOn(bcrypt, 'compare').mockResolvedValue(true); // stubs method,
preserves module
jest.mock('../lib/prisma'); // hoists and
replaces entire module
```

Integration Testing with Supertest

Supertest boots an Express application over ephemeral loopback ports to issue real HTTP requests without requiring an explicit `app.listen()` call. Tests status codes, headers, response bodies, and real database state end-to-end.

```
import request from 'supertest'; import { app } from '../app';
const res = await request(app).get('/health').expect(200); // issues HTTP
GET request
expect(res.body.status).toBe('healthy');
```

Test Lifecycle Hooks

Manage test states using built-in lifecycle hooks: `beforeAll/afterAll` handle suite-level migrations or connection pools, while `beforeEach/afterEach` isolate test data by resetting mocks or rolling back transactions.

```
beforeAll(async () => await prisma.$connect()); // connect once before
tests start
afterEach(async () => jest.clearAllMocks()); // isolate test runs
afterAll(async () => await prisma.$disconnect()); // close handles to let
Jest exit cleanly
```

Dockerfile Anatomy & Layer Caching

Each instruction in a Dockerfile generates an immutable, cached layer. Copy `package*.json` and install dependencies *before* copying your source code to avoid re-running expensive install steps when only application files change.

```
COPY package*.json ./
RUN npm ci --only=production # cached if package files do not change
COPY . .                    # invalidates cache only when source code
                             changes
```

Multi-Stage Docker Builds

Compile TypeScript and generate dependencies inside a heavy "builder" image, then copy only the compiled `/dist` directory and production dependencies into a lightweight runtime image (shrinking image size by ~70%).

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY . .
RUN npm ci && npx tsc # builds /app/dist

FROM node:20-alpine AS production
WORKDIR /app
COPY --from=builder /app/dist ./dist # copy only compiled JS into runtime
image
CMD ["node", "dist/index.js"]
```

.dockerignore

Prevents local `node_modules`, `.env` files, build caches, and test suites from being copied into the Docker daemon build context.

```
node_modules
.env
.git
dist
```

Docker Compose Healthchecks & Volumes

Compose runs multi-container setups using shared networks so containers can resolve each other by service name. Use `condition: service_healthy` on `depends_on` to ensure your app waits for dependent services like Postgres to be fully operational before booting.

```

services:
  app:
    depends_on:
      postgres: { condition: service_healthy } # waits for DB readiness
  probe
  postgres:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"] # verifies DB accepts
      queries

```

GitHub Actions CI Workflow

Automates testing pipelines on pushes and pull requests using GitHub-hosted runners. Use `npm ci` for clean, deterministic builds from `package-lock.json`, spin up ephemeral Docker service containers for integration databases, and run migrations via `prisma migrate deploy`.

```

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:16-alpine, ports: ['5432:5432'] } # CI
    service container
    steps:
      - uses: actions/checkout@v4
      - run: npm ci # deterministic install
      - run: npx prisma migrate deploy # applies migrations without prompts
      - run: npm run test:ci # executes test suites

```

===== DAY 1 — NODE RUNTIME

1. NODE.JS Node.js = V8-based JS runtime outside browser; designed for I/O-heavy, concurrent server workloads. Think: JS + V8 + libuv + OS APIs → server. `process.version`
2. V8 V8 executes JavaScript; parses/JIT-compiles hot code to machine code and manages Stack + Heap. Call Stack = executing functions; Heap = objects/arrays/closures. `node --prof app.js`
3. CALL STACK Tracks currently executing synchronous JS; too much recursion → stack overflow. `function f(){ f() }` → `RangeError: Maximum call stack size exceeded`
4. HEAP + GC Objects live in heap; GC removes unreachable objects, while retained references can cause memory leaks/OOM. `const obj = {};` // object in heap; `v8.writeHeapSnapshot()` helps investigate leaks.

5. libuv libuv provides Node's event loop, OS async I/O and a worker thread pool for operations needing threads. fs/crypto/zlib/dns.lookup → thread pool; TCP/HTTP + dns.resolve → OS/c-ares async.
6. LIBUV THREAD POOL Default pool = 4 threads; fs, crypto, zlib and dns.lookup can occupy it and queue excess work. process.env.UV_THREADPOOL_SIZE = '8'; // set before modules using the pool
7. EVENT LOOP Node runs JS on one main thread and uses the event loop to process async callbacks without blocking on I/O. Phases: timers → pending callbacks → idle/prepare → poll → check → close callbacks.
8. process.nextTick() Runs with highest microtask priority, before Promise microtasks / next event-loop phase. process.nextTick(() => console.log('nextTick'));
9. PROMISE MICROTASK Promise.then/catch/finally callbacks run after current synchronous JS and after nextTick queue. Promise.resolve().then(() => console.log('promise'));
10. setImmediate() Runs in the check phase; especially useful after I/O and for yielding between recursive async work. setImmediate(() => console.log('immediate'));
11. setTimeout(0) Schedules callback for the timers phase; "0" means minimum delay, NOT immediate execution. setTimeout(() => console.log('timer'), 0);
12. nextTick vs setImmediate nextTick has higher priority and can starve the event loop if recursively scheduled. setImmediate(() => {}); // yields back to I/O; safer for repeated async work.
13. WORKER THREADS Worker Threads = parallel JS execution for CPU-heavy work; each worker has its own V8 isolate. new Worker('./worker.js'); worker.postMessage(data); worker.on('message', result => {});
14. WORKER COMMUNICATION Workers communicate using postMessage/message events; SharedArrayBuffer + Atomics can share memory. const shared = new SharedArrayBuffer(4); // shared memory between workers
15. WHEN NOT TO USE WORKERS Don't use workers just because something is async; normal network/file I/O is already handled asynchronously. Use workers for CPU: image processing, hashing, compression, ML, huge JSON parsing.
16. CLUSTER Cluster creates multiple Node processes, each with separate V8/heap/event loop, while sharing the same server port. cluster.fork(); // scale HTTP server across CPU cores
17. WORKER vs CLUSTER Worker = threads inside one process for CPU parallelism; Cluster = separate processes for HTTP/process scaling. Worker → shared memory possible; Cluster → isolated memory + IPC; Redis needed for shared application state.
18. COMMONJS CJS uses require/module.exports, loads synchronously and caches modules after first load. const x = require('./x'); module.exports = { x };
19. CJS EXPORT DIFFERENCE exports initially points to module.exports; replacing exports breaks the reference. exports.x = 1; // works module.exports = { x: 1 }; // works exports = { x: 1 }; // does NOT change exported object

20. **MODULE RESOLUTION** `require('./x')` searches files/extensions then directories/node_modules according to Node's resolution rules. `require('lodash')`; // searches node_modules hierarchy
21. **ESM** ESM uses import/export, is statically analyzable, supports top-level await and modern module semantics. `import fs from 'node:fs'; export const x = 1;`
22. **CJS vs ESM** CJS → `require/module.exports`; ESM → `import/export`; ESM supports top-level await and static analysis. `package.json` → `{ "type": "module" }` // enables ESM .js semantics
23. **ESM** `__dirname` ESM doesn't provide `__dirname/__filename` directly; derive them from `import.meta.url`. `const __dirname = dirname(fileURLToPath(import.meta.url));`
24. **CIRCULAR DEPENDENCY** A imports B while B imports A → partially initialized exports can be observed and produce undefined values. Avoid by extracting shared logic into a third module.
25. **fs MODULE** fs provides filesystem operations; prefer `async/promises` APIs for servers to avoid blocking the event loop. `import { readFile } from 'node:fs/promises'; const data = await readFile('a.txt');`
26. **fs SYNC vs ASYNC** `readFileSync` blocks the JS thread; `readFile/readFile` from `fs/promises` lets Node continue handling other work. `await fs.readFile('big.txt');` // async `fs.readFileSync('big.txt');` // blocks
27. **path MODULE** path safely constructs/parses OS-specific filesystem paths instead of manually concatenating strings. `path.join(__dirname, 'uploads', 'avatar.png');`
28. **STREAMS** Streams process data chunk-by-chunk instead of loading the entire payload into memory. `fs.createReadStream('4GB.mp4').pipe(res);`
29. **STREAM TYPES** Readable = produces data; Writable = consumes; Duplex = both; Transform = modifies data while flowing. `readable.pipe(transform).pipe(writable);`
30. **BACKPRESSURE** When writable is slower, producer must slow down; `write()` returning false means buffer is full. `if (!writable.write(chunk)) readable.pause(); writable.once('drain', () => readable.resume());`
31. **pipe()** `pipe()` connects streams and automatically handles backpressure/error flow better than manually buffering. `readable.pipe(writable);`
32. **EVENTEMITTER** `EventEmitter` implements pub/sub: listeners subscribe to named events and emit triggers them. `const e = new EventEmitter(); e.on('done', fn); e.emit('done', data);`
33. **on vs once** `on()` remains registered for every emission; `once()` automatically removes itself after first execution. `emitter.once('connected', () => console.log('only once'));`
34. **removeListener** Remove a registered listener when it should no longer receive events. `emitter.off('event', handler);` // same idea as `removeListener`
35. **CRYPTO HASH** Hashing is one-way transformation; same input → same digest, useful for integrity but NOT suitable alone for passwords. `crypto.createHash('sha256').update(data).digest('hex');`
36. **RANDOM BYTES / UUID** `randomBytes` gives cryptographically secure random data; `randomUUID` gives UUID v4 identifiers. `crypto.randomBytes(32).toString('hex');`; `crypto.randomUUID();`

37. HMAC HMAC combines hashing + secret key to verify authenticity and integrity of data.
`crypto.createHmac('sha256', secret).update(payload).digest('hex');`
38. timingSafeEqual Use `timingSafeEqual` when comparing secrets/signatures to reduce timing-attack leakage. `crypto.timingSafeEqual(Buffer.from(a), Buffer.from(b));`
39. PROCESS process represents the current Node process and connects application code with the OS/environment. `process.env.PORT`; `process.pid`; `process.cwd()`; `process.exit(1)`;
40. ENVIRONMENT VARIABLES Secrets/config belong in `process.env`, not source code. `const port = Number(process.env.PORT) || 3000;`
41. PROCESS SIGNALS SIGTERM/SIGINT indicate shutdown; close server/resources before exiting.
`process.on('SIGTERM', () => server.close(() => process.exit(0)));`
42. CHILD PROCESS Child processes execute external commands/programs outside the current Node process. `exec('ls -la', (err, stdout) => console.log(stdout));`
43. BUFFER Buffer represents raw binary bytes; used for files, sockets, streams and uploaded data.
`const buf = Buffer.from('hello'); buf.toString(); // 'hello'`
44. MEMORY MODEL TRIGGER Stack → current function execution; Heap → objects; libuv → async I/O/thread pool; Event Loop → callback scheduling. CPU-heavy JS blocks event loop → Worker Thread; HTTP scaling → Cluster.
45. CORE DAY-1 MENTAL MODEL Request arrives → Node main JS thread handles orchestration → libuv/OS handles waiting → callback returns to event loop. CPU work is the exception: it must be moved away from the main JS thread.

===== DAY 2 — HTTP + EXPRESS + ASYNC

1. HTTP REQUEST Request = method + URL + headers + optional body; server interprets these to decide what to do. `POST /users HTTP/1.1 Content-Type: application/json {"name": "Avishek"}`
2. HTTP METHODS GET=read, POST=create, PUT=replace, PATCH=partial update, DELETE=remove. GET/PUT/DELETE are generally idempotent; POST is not.
3. HTTP RESPONSE Response = status line + headers + body; status tells client the result category.
`res.status(201).json({ id: 1 });`
4. STATUS CODES 2xx success, 3xx redirect, 4xx client error, 5xx server error. 200 OK | 201 Created | 204 No Content | 400 Bad Request | 401 Unauthenticated | 403 Forbidden | 404 Not Found | 409 Conflict | 429 Too Many | 500 Server Error
5. 401 vs 403 401 = "I don't know who you are"; 403 = "I know you, but you cannot do this." return `res.status(401).json(...); // auth missing/invalid` return `res.status(403).json(...); // permission denied`
6. CORS CORS is a browser security mechanism controlling cross-origin JS requests; complex requests trigger OPTIONS preflight. `app.use(cors({ origin: 'https://frontend.com', credentials: true`

```
});
```

7. COOKIES Cookies are browser-managed key/value data; HttpOnly prevents JS access, Secure requires HTTPS, SameSite controls cross-site sending. `res.cookie('refreshToken', token, { httpOnly:true, secure:true, sameSite:'strict' });`
8. EXPRESS Express is a thin web framework around Node's http server; it mainly provides routing + middleware abstractions. `const app = express(); app.listen(3000);`
9. MIDDLEWARE Middleware receives req/res/next and can modify request, respond, or pass control to the next layer. `app.use((req,res,next) => { req.start = Date.now(); next(); });`
10. next() next() moves request execution forward; forgetting next() or a response causes the request to hang. `app.use((req,res,next) => { console.log('A'); next(); });`
11. MIDDLEWARE ORDER Middleware executes in registration order, so body parser/auth/logging must be registered before routes needing them. `app.use(express.json()); app.use(auth); app.use('/api', router);`
12. ERROR MIDDLEWARE Express identifies error middleware by FOUR parameters: (err, req, res, next), and it should be registered last. `app.use((err, req, res, next) => res.status(500).json({ error: err.message }));`
13. req.params Route parameters identify a resource. `GET /users/:id → req.params.id`
14. req.query Query parameters control filtering, pagination, sorting etc. `GET /users?page=2&limit=20 → req.query.page`
15. req.body Body contains client payload and requires a parser such as `express.json()`. `app.use(express.json()); req.body.email`
16. req.headers Headers carry metadata such as authorization/content type. `req.headers.authorization`
`req.get('Content-Type')`
17. req.ip Provides client/proxy IP depending on proxy configuration. `app.set('trust proxy', 1); req.ip`
18. RESPONSE APIs `json()` sends JSON, `status()` sets status, `cookie()` sets cookie, `send()` sends body, `end()` closes response. `res.status(201).json(data); res.cookie('x', token); res.status(204).end();`
19. NEVER RESPOND TWICE After sending a response, return/stop execution or later code may try to send another response. `if (!user) return res.status(404).json({ error:'Not found' });`
20. `express.Router()` Router groups related endpoints into modular route files. `const router = express.Router(); router.get('/:id', controller.get);`
21. `async/await` `async` function returns Promise; `await` pauses that `async` function until Promise settles without blocking the Node event loop. `const user = await getUser(id);`
22. PROMISE.ALL Runs independent promises concurrently and rejects immediately when one rejects. `const [user, posts] = await Promise.all([getUser(), getPosts()]);`
23. PROMISE.ALLSETTLED Waits for every promise and returns success/failure for each; useful when partial failures are acceptable. `const results = await Promise.allSettled(tasks);`

24. **PROMISE.RACE** Settles when the first promise settles, whether success or failure; useful for timeout/hedging patterns. `await Promise.race([request(), timeout(3000)]);`
25. **PROMISE.ANY** Returns first fulfilled promise and ignores rejected promises until all fail. `await Promise.any([fetch(primary), fetch(backup)]);`
26. **for await...of** Iterates async iterables sequentially and waits for each yielded value. `for await (const chunk of stream) { process(chunk); }`
27. **forEach ASYNC TRAP** `Array.forEach` does not await async callbacks; use `for...of` for sequential work or `Promise.all` for parallel work. `await Promise.all(users.map(async u => save(u)));`
28. **OPERATIONAL ERROR** Expected runtime failure: invalid input, DB unavailable, resource missing etc.; handle and return controlled response. `throw new NotFoundError('User');`
29. **PROGRAMMER ERROR** Unexpected bug: undefined access, broken invariant, coding mistake; fix code rather than hiding it. `user.profile.name` // if profile unexpectedly undefined
30. **CUSTOM ERROR** Create typed errors carrying status/code/isOperational so global handler knows how to respond. `class NotFoundError extends AppError { constructor(resource:string){ super(`${resource} not found`,404); } }`
31. **ASYNC WRAPPER** Wrap async route handlers so Promise rejection automatically reaches `next(err)`. `const asyncWrapper = fn => (req,res,next) => Promise.resolve(fn(req,res,next)).catch(next);`
32. **GLOBAL ERROR HANDLER** Central place converts application errors into consistent HTTP responses and logs unexpected errors. `app.use((err,req,res,next) => { res.status(err.statusCode ?? 500).json({ success:false,error:err.message }); });`
33. **404 FALLBACK** If no route matched, create a route-not-found error before global error middleware. `app.use((req,res,next) => next(new NotFoundError(req.originalUrl)));`
34. **uncaughtException** Synchronous error escaped all try/catch; process may be corrupted → log and terminate, letting supervisor restart. `process.on('uncaughtException', err => { logger.error(err); process.exit(1); });`
35. **unhandledRejection** Promise rejection has no handler; treat it as process-level failure and shut down gracefully where appropriate. `process.on('unhandledRejection', reason => server.close(() => process.exit(1)));`
36. **GRACEFUL SHUTDOWN** Stop accepting requests, finish current work, close DB/Redis/server connections, then exit. `process.on('SIGTERM', async () => { await prisma.$disconnect(); server.close(() => process.exit(0)); });`
37. **KEEP-ALIVE** HTTP/1.1 can reuse TCP connections rather than reconnecting for every request. `server.keepAliveTimeout = 5000;`
38. **COMPRESSION** gzip/Brotli reduces HTTP response size for text/JSON payloads. `app.use(compression());`
39. **CACHE-CONTROL** Controls client/proxy caching behavior. `res.set('Cache-Control', 'public, max-age=86400');`

40. app.locals vs res.locals app.locals = shared app-wide data; res.locals = data belonging only to the current request. app.locals.version = '1'; res.locals.user = req.user;
41. util.promisify Converts callback-style APIs to Promise style, though native fs/promises is preferred where available. const readFile = promisify(fs.readFile);
42. DAY-2 REQUEST FLOW Client → Node http.Server → Express middleware stack → router → controller → response. Error anywhere → next(err) → 4-arg global error handler.

===== DAY 3 — TYPESCRIPT + ZOD + DB + PRISMA

1. tsconfig Controls TypeScript strictness, target/module, source/output directories, aliases and compiler behavior. "strict": true, "target": "ES2022", "rootDir": "./src", "outDir": "./dist"
2. strict:true Enables strictNullChecks, noImplicitAny, strictFunctionTypes, strictPropertyInitialization and more. "strict": true // backend safety net
3. noUncheckedIndexedAccess Array/object indexed access becomes potentially undefined, forcing explicit checks. const user = users[0]; // User | undefined
4. noImplicitReturns Every code path must return when function expects a value. function getUser(id:string): User { if (...) return user; throw new Error(); }
5. noFallthroughCasesInSwitch Prevents accidental switch fallthrough. switch(role){ case 'admin': return true; case 'user': return false; }
6. rootDir / outDir TypeScript reads source from rootDir and emits compiled JS to outDir. src/index.ts → dist/index.js
7. PATH ALIASES Aliases shorten imports but tsconfig paths alone don't teach Node runtime how to resolve them. "paths":{"@/":["src/"]}
8. esModuleInterop Makes CommonJS packages easier to consume with default imports. import express from 'express';
9. sourceMap Maps compiled JS back to TS source so production stack traces point to original TS lines. "sourceMap": true
10. declaration merging Extends existing types such as Express.Request instead of casting everything to any. declare global { namespace Express { interface Request { user?: User; requestId:string } } }
11. GENERICS Generics allow reusable type-safe abstractions while preserving the concrete type. interface Repository { findById(id:string): Promise<T|null>; }
12. UTILITY TYPES Partial=all optional; Pick=select fields; Omit=remove fields; ReturnType=function return; Awaited=unwrap Promise. type UpdateUser = Partial; type PublicUser = Pick<User,'id'|'name'>;

13. DISCRIMINATED UNION Union variants share a literal field, allowing TypeScript to narrow safely.
type Result = {success:true;data:User} | {success:false;error:string};
14. TYPE GUARD A type guard narrows unknown/union values to a specific type. function
isAppError(e:unknown): e is AppError { return e instanceof AppError; }
15. ZOD Zod validates untrusted runtime input AND generates its TypeScript type from the same
schema. const UserSchema = z.object({ email:z.string().email(), age:z.number().int() }); type User
= z.infer;
16. ZOD BASIC TYPES object/string/number/enum/array form common API schemas. z.object({
role:z.enum(['USER','ADMIN']), tags:z.array(z.string()) });
17. parse vs safeParse parse throws on invalid input; safeParse returns {success,data/error} and is
better when you want controlled handling. const result = schema.safeParse(req.body);
18. refine refine adds custom/cross-field validation. z.object({pass:z.string(),confirm:z.string()})
.refine(x => x.pass === x.confirm);
19. transform transform validates then converts data into another representation. z.string().transform(v
=> v.trim().toLowerCase());
20. extend/pick/omit/partial Reuse schemas instead of duplicating validation definitions. const
UpdateSchema = CreateSchema.partial(); const PublicSchema =
UserSchema.pick({id:true,name:true});
21. ZOD EXPRESS MIDDLEWARE Validate at the API boundary so controllers receive trusted typed
data. const result = schema.safeParse(req.body); if (!result.success) return
res.status(400).json(result.error);
22. SQL SELECT SELECT reads rows; WHERE filters; ORDER BY sorts; LIMIT/OFFSET paginates.
SELECT * FROM users WHERE role='ADMIN' ORDER BY created_at DESC LIMIT 20;
23. INSERT/UPDATE/DELETE INSERT creates, UPDATE modifies, DELETE removes records. INSERT
INTO users(name) VALUES('Avis'); UPDATE users SET name='A' WHERE id=1;
24. INNER JOIN Returns only rows having matching records on both sides. SELECT u.name,p.title
FROM users u INNER JOIN posts p ON p.user_id=u.id;
25. LEFT JOIN Keeps every row from the left table even if no matching right row exists. SELECT
u.name,p.title FROM users u LEFT JOIN posts p ON p.user_id=u.id;
26. PRIMARY KEY Uniquely identifies a row and cannot be duplicated/null. id UUID PRIMARY KEY
27. FOREIGN KEY Connects a child row to a parent row and enforces referential integrity. user_id UUID
REFERENCES users(id)
28. CONSTRAINTS DB constraints enforce correctness even if application validation is bypassed.
UNIQUE(email); CHECK(age >= 18); NOT NULL
29. INDEX Indexes speed reads/filtering but consume storage and make writes slightly more expensive.
CREATE INDEX idx_users_email ON users(email);

- 30. 1:1 RELATION One record maps to one record; FK + UNIQUE typically enforces it. `userId UUID
UNIQUE REFERENCES users(id)`
- 31. 1:N RELATION One parent has many children; child stores parent's FK. `posts.authorId → users.id`
- 32. M:N RELATION Many-to-many uses a junction table containing both foreign keys.
`post_tags(post_id, tag_id)`
- 33. ACID TRANSACTION Atomicity, Consistency, Isolation, Durability → all related writes succeed or rollback together. `BEGIN; UPDATE ...; UPDATE ...; COMMIT; // failure → ROLLBACK`
- 34. SAVEPOINT Partial rollback inside a transaction. `SAVEPOINT s1; ... ROLLBACK TO SAVEPOINT s1;`
- 35. N+1 QUERY Fetch N records, then run one related query per record → 1 + N queries. Fix: `JOIN/include/batching/DataLoader` instead of querying inside a loop.
- 36. CONNECTION POOL Reuse a fixed set of DB connections instead of creating one TCP connection per request. `const pool = new Pool({ connectionString: process.env.DATABASE_URL });`
- 37. POOL MENTAL MODEL Request borrows connection → query → releases connection → next request reuses it. `const client = await pool.connect(); try { ... } finally { client.release(); }`
- 38. PRISMA SCHEMA `schema.prisma` defines `datasource`, `generator`, `models`, `fields`, `relations`, `indexes` and `constraints`. `model User { id String @id @default(uuid()) email String @unique }`
- 39. PRISMA FIELD FEATURES `@id` primary key, `@default` default value, `@unique` unique constraint, `@updatedAt` auto timestamp, `@map` DB name. `createdAt DateTime @default(now()) updatedAt DateTime @updatedAt`
- 40. PRISMA RELATIONS Prisma models represent 1:1, 1:N and M:N relationships. `posts Post[]; profile Profile?; tags Tag[] @relation("PostTags")`
- 41. PRISMA MIGRATE DEV Development: creates migration SQL, applies it locally and regenerates Prisma client. `npx prisma migrate dev --name add_avatar`
- 42. PRISMA MIGRATE DEPLOY Production/CI: applies existing migration files only; does NOT generate new migrations. `npx prisma migrate deploy`
- 43. PRISMA CREATE Creates one record; select only fields that should leave DB.
`prisma.user.create({data:{email},select:{id:true,email:true}})`
- 44. `createMany` Batch inserts multiple records and can skip duplicates.
`prisma.tag.createMany({data:tags,skipDuplicates:true});`
- 45. `findUnique` Fetches one record by primary/unique field and returns object/null.
`prisma.user.findUnique({where:{id}});`
- 46. `findUniqueOrThrow` Same lookup but throws if missing. `prisma.user.findUniqueOrThrow({where:{id}});`
- 47. `findMany` Fetch multiple rows with filtering, sorting and pagination. `prisma.user.findMany({where:{role:'USER'},orderBy:{createdAt:'desc'}});`

48. update Updates a record matching a unique selector. `prisma.user.update({where:{id},data:{name:'Avis'}});`
49. upsert If record exists update it; otherwise create it. `prisma.user.upsert({where:{email},update:{name},create:{email,name}});`
50. delete Permanently deletes record; soft delete instead sets `deletedAt`. `prisma.user.delete({where:{id}});`
51. SELECT vs INCLUDE select controls returned fields; include loads related records; don't over-fetch unnecessary columns. `prisma.user.findUnique({ where:{id}, select:{id:true,name:true} });`
52. where Filters rows using equality, ranges, contains, OR/NOT etc. `where:{createdAt:{gte:start},role:'USER'}`
53. orderBy Sorts query result. `orderBy:[{createdAt:'desc'},{name:'asc'}]`
54. take / skip Offset pagination uses take + skip. `take:20, skip:40`
55. CURSOR PAGINATION Uses a stable unique cursor instead of large OFFSETs; better for deep/large datasets. `take:20, skip:1, cursor:{id:lastId}`
56. PRISMA TRANSACTION `$transaction` groups operations atomically; either all succeed or rollback. `await prisma.$transaction([ prisma.user.update(...), prisma.account.update(...) ]);`
57. INTERACTIVE TRANSACTION Use callback when later operations depend on earlier results. `await prisma.$transaction(async tx => { const user = await tx.user.create(...); await tx.profile.create({data:{userId:user.id}}); });`
58. NESTED WRITE Create/update related records through Prisma in one logical operation. `prisma.user.create({ data:{email, profile:{create:{bio:'hello'}}} });`
59. GROUP BY Groups rows for aggregation; HAVING filters aggregate results. `SELECT user_id,COUNT() FROM posts GROUP BY user_id HAVING COUNT() > 5;`
60. CTE WITH creates a named temporary result used by a larger query, improving readability for complex SQL. `WITH active AS (SELECT * FROM users WHERE is_active=true) SELECT * FROM active;`
61. WINDOW FUNCTION Calculates over related rows without collapsing them like GROUP BY. `RANK() OVER(PARTITION BY department ORDER BY salary DESC)`
62. `$queryRaw` Use parameterized tagged template SQL when Prisma can't express the query. `await prisma.queryRaw`SELECT * FROM users WHERE id = ${userId}`;`
63. `$executeRaw` Use for raw INSERT/UPDATE/DELETE operations and get affected-row count. `await prisma.$executeRawUPDATE sessions SET valid=false WHERE expires_at < NOW();`

===== DAY 4 — AUTH + DEBUGGING

1. BCrypt bcrypt is intentionally slow password hashing with configurable cost factor + random salt; never store plaintext passwords. `const hash = await bcrypt.hash(password, 12); const ok = await bcrypt.compare(password, hash);`
2. BCrypt SALT Same password produces different bcrypt hashes because each hash contains a random salt. `bcrypt.hash('same',12) !== bcrypt.hash('same',12)`
3. BCrypt COST Higher cost = more computation = harder brute force but slower login/register; tune based on acceptable latency. `bcrypt.hash(password, 12);`
4. PASSWORD LOGIN SECURITY Don't reveal whether email exists; use generic invalid-credentials response and same bcrypt path to reduce enumeration/timing leakage. `const valid = user ? await bcrypt.compare(password,user.password) : false;`
5. JWT JWT = header.payload.signature; payload is readable, signature provides integrity/authenticity — JWT is NOT encryption. `jwt.sign({sub:user.id,role:user.role}, secret, {expiresIn:'15m'});`
6. JWT VERIFY Always verify signature + expiry + expected algorithm/issuer/audience; never trust `jwt.decode()` for authentication. `jwt.verify(token, secret, {algorithms:['HS256'],issuer:'myapp'});`
7. JWT CLAIMS sub=user identity, exp=expiry, iat=issued time, jti=token ID; custom claims can contain role/type etc. `{ sub:'user-1', role:'ADMIN', type:'access', exp:... }`
8. HS256 Symmetric: same secret signs + verifies; simple but every verifying service knows the signing secret. `jwt.sign(payload, SHARED_SECRET, {algorithm:'HS256'});`
9. RS256 Asymmetric: auth service signs with private key; other services verify with public key without being able to forge tokens. `jwt.verify(token, publicKey, {algorithms:['RS256']});`
10. JWKS Auth service publishes public keys through JWKS so distributed services can fetch verification keys and support key rotation. `GET /.well-known/jwks.json`
11. ACCESS TOKEN Short-lived/stateless token sent with API requests; server verifies signature rather than DB lookup. Authorization: Bearer
12. REFRESH TOKEN Long-lived token used only to obtain new access tokens; store its hash in DB and preferably raw token in HttpOnly cookie. `res.cookie('refresh', token, {httpOnly:true,secure:true,sameSite:'strict'});`
13. TOKEN ROTATION Every refresh issues a new refresh token and invalidates the old one; reuse of an invalidated token can indicate theft. `oldRefresh → invalidate → newRefresh`
14. REFRESH TOKEN HASHING Store hash, not raw refresh token; DB leak should not immediately give attackers usable refresh credentials. `sha256(refreshToken) → DB lookup`
15. TOKEN STORAGE Access token can live in memory; refresh token should be HttpOnly + Secure cookie to prevent JS stealing it via XSS. `document.cookie` // cannot read HttpOnly cookie
16. JWT ERRORS Expired → `TokenExpiredError`; malformed/signature invalid → `JsonWebTokenError`; not-yet-valid → `NotBeforeError`. `try { jwt.verify(token,secret) } catch(err) { /* map to 401 */ }`

17. AUTHENTICATION Authentication answers "Who are you?" — verify token/session and attach identity to request. `req.user = verifiedPayload; next();`
18. AUTHORIZATION Authorization answers "What are you allowed to do?" — check role/permission after authentication. `if (!req.user.permissions.includes('write:jobs')) return res.status(403).end();`
19. RBAC Role-Based Access Control maps roles → permissions, e.g. ADMIN → delete/write/read. `const rolePermissions = { ADMIN:['read','write','delete'], USER:['read'] };`
20. AUTH MIDDLEWARE Extract Bearer token → verify → attach user → next; missing/invalid token → 401. `const token = req.headers.authorization?.split(' ')[1];`
21. PERMISSION MIDDLEWARE Check required permission after authentication; authenticated but unauthorized → 403. `requirePermission('delete:jobs')`
22. OWNERSHIP CHECK Authorization may require resource ownership, not just role. if `(job.createdById !== req.user.id)` throw new `ForbiddenError()`;
23. VS CODE DEBUGGER `launch.json` defines how VS Code starts/attaches to Node/TS debugging. `"program": "${workspaceFolder}/src/index.ts"`
24. BREAKPOINTS Line breakpoint pauses execution; conditional breakpoint pauses only when expression is true; logpoint logs without stopping. `if (user.id === targetId) // conditional breakpoint`
25. STEP CONTROLS Step Over = execute current line; Step Into = enter function; Step Out = finish current function. F10 = over | F11 = into | Shift+F11 = out
26. CALL STACK Shows nested execution path so you can trace "who called this function?" controller → service → repository → prisma
27. WATCH Evaluate variables/expressions while paused to inspect changing state. `user.permissions.includes('write')`
28. ASYNC STACK TRACE `async/await` preserves useful logical call-chain information across asynchronous boundaries. `await service.createUser()` → debugger can follow async caller chain.
29. NODE STACK TRACE Read from error message → file/line → call stack bottom-to-top to understand where failure originated. `TypeError ... at UserService.create (user.service.ts:42)`
30. MISSING AWAIT Without `await`, you get Promise instead of resolved value; downstream code may behave incorrectly. `const user = getUser(); // Promise` `const user = await getUser(); // User`
31. CODE READING ORDER For unfamiliar backend feature: Routes → Controller → Service → Repository → DB. Route tells entry point; service tells business rules; repository tells actual DB operation.
32. READ PRISMA SCHEMA FIRST Schema reveals entities, relations, constraints and available data before reading hundreds of lines of service code. User 1:N Post → expect `post.authorId` and related queries.
33. FOLLOW TYPES Types/DTOs reveal what enters/exits functions and often explain behavior faster than implementation. `CreateUserDto` → `Service.createUser(CreateUserDto)`

34. GREP ERROR Search exact error message to find where it is created, thrown, transformed or handled. `grep -R "Invalid credentials" src/`
35. READ TESTS FIRST Tests reveal expected behavior, edge cases and errors; implementation explains HOW, tests explain WHAT. `it('throws UnauthorizedError for invalid password', ...)`

===== DAY 5 — ARCHITECTURE + LOGGING + SECURITY + UPLOAD

1. FOUR-LAYER ARCHITECTURE Request → Route → Controller → Service → Repository → DB; each layer has ONE responsibility. Route=wire | Controller=HTTP | Service=business | Repository=DB
2. ROUTE Defines path + middleware + delegates to controller; no business logic/DB calls.
`router.post('/', authenticate, validate(schema), controller.create);`
3. CONTROLLER Reads req, calls service with plain data, formats HTTP response; NEVER Prisma/business logic. `const result = await service.create(req.body, req.user.id);
res.status(201).json(result);`
4. SERVICE Contains business rules/orchestration and calls repositories; should know nothing about req/res/HTTP. `if (activeCount >= 50) throw new ConflictError(...);`
5. REPOSITORY Owns database queries; service talks to repository instead of directly talking to Prisma. `class JobRepository { findById(id){ return prisma.job.findUnique({where:{id}}); } }`
6. REPOSITORY PATTERN Service depends on repository interface/contract, making DB implementation replaceable and services easy to unit test. `interface JobRepository { findById(id:string): Promise<Job|null>; }`
7. DEPENDENCY INJECTION Dependencies are passed into a class/function instead of being hard-imported internally. `constructor(private repo: JobRepository) {}`
8. DI BENEFIT Production can use PrismaRepository; tests can inject MockRepository without a real DB. `new JobService(mockRepo)`
9. SOLID — SINGLE RESPONSIBILITY A class/function should have one reason to change; controller shouldn't become a business-logic container. Controller changes for HTTP; Service changes for business rules.
10. SOLID — DEPENDENCY INVERSION High-level service depends on abstraction, not concrete Prisma implementation. `Service → IJobRepository ← PrismaJobRepository`
11. OPEN/CLOSED Extend behavior without repeatedly modifying stable business code. `new EmailNotifier/SmsNotifier implementations → JobService unchanged.`
12. LISKOV Replacement implementation must honor the same contract/behavior expected by consumers. `MockJobRepository` must behave like real repository.
13. INTERFACE SEGREGATION Prefer small focused interfaces over giant interfaces. `IReadRepository + IWriteRepository > IRepository with 20 methods`

14. **FAT CONTROLLER** Controller containing business logic/Prisma queries = architecture violation.
router → controller → prisma ❌ router → controller → service → repository → prisma ✅
15. **FAT SERVICE** Service directly reading req/res means HTTP concerns leaked into business layer.
service.create(req.body) ❌ service.create(data) ✅
16. **MISSING TRANSACTION** Two related writes without transaction can leave inconsistent partial state.
await debit(); await credit(); // credit fails → broken state \$transaction([...]) // atomic
17. **MAGIC STRINGS/NUMBERS** Repeated raw values create inconsistency; centralize constants/enums. JobStatus.CLOSED MAX_ACTIVE_JOBS = 50
18. **DUPLICATE VALIDATION** Don't validate the same business rule independently in controller/service/repository; establish clear boundaries. Zod → input shape; Service → business rule; DB → data integrity.
19. **STRUCTURED LOGGING** console.log is unstructured; production logging should be searchable JSON with levels/context. logger.info({userId,requestId}, 'User created');
20. **PINO** pino produces structured JSON logs with levels and child loggers, making logs machine-queryable. const logger = pino(); logger.error({err}, 'DB failed');
21. **LOG LEVELS** trace/debug/info/warn/error/fatal let production systems filter signal from noise. logger.warn({userId}, 'Rate limit approaching');
22. **CHILD LOGGER** Attach persistent context to all logs produced by a request/component. const log = logger.child({requestId}); log.info('Started'); log.error({err},'Failed');
23. **CORRELATION ID** Generate/propagate a request ID so every log from that request can be traced across layers/services. const requestId = req.headers['x-correlation-id'] ?? randomUUID(); req.log = logger.child({requestId});
24. **NEVER LOG** Never log passwords, JWTs, refresh tokens, API keys, secrets or unnecessary PII. logger.info({userId}, 'login'); // okay logger.info({password}, 'login'); // NEVER
25. **HELMET** Helmet sets security response headers such as CSP, HSTS, X-Frame-Options and X-Content-Type-Options. app.use(helmet());
26. **CSP** Content-Security-Policy restricts which scripts/resources browsers may execute/load, reducing XSS impact. Content-Security-Policy: script-src 'self'
27. **HSTS** HSTS tells browsers to use HTTPS for future requests to the domain. Strict-Transport-Security: max-age=31536000
28. **X-FRAME-OPTIONS** Prevents clickjacking by controlling whether the page can be embedded in an iframe. X-Frame-Options: DENY
29. **RATE LIMITING** Limit requests per client/IP/user to reduce brute force/abuse; Redis store enables shared limits across instances. rateLimit({windowMs:60_000,max:100})
30. **REDIS RATE LIMIT** Multiple Node instances need shared counters; local in-memory counters would differ per instance. Redis INCR key → atomic shared count

31. IDOR Never trust an ID from URL/body to decide authorization; verify the current user owns/can access the resource. GET /orders/123 → check order.userId === req.user.id
32. INPUT SANITIZATION Treat all client input as untrusted; validate structure and sanitize content where HTML/user-generated markup is allowed. `schema.parse(req.body)`
33. XSS Attacker injects JS into content; mitigate with output escaping/CSP/sanitization. `sanitize(userHtml)` + CSP
34. CSRF Attacker tricks browser into sending authenticated cookie-based requests; mitigate with SameSite cookies/CSRF tokens where needed. `SameSite:'strict'`
35. SQL INJECTION Never concatenate user input into raw SQL; use Prisma parameterization/tagged SQL. `prisma.queryRaw`SELECT * FROM users WHERE id = {userId}` // safe`
36. FILE UPLOAD Use multipart parser such as multer; enforce size/count/type limits before processing. `const upload = multer({limits:{fileSize:510241024}});`
37. MULTER diskStorage diskStorage writes uploads to local filesystem; convenient locally but problematic with containers/multiple instances. `multer({storage:multer.diskStorage({...}})`
38. MULTER memoryStorage memoryStorage places uploaded file in req.file.buffer; useful when immediately sending to S3/cloud storage. `multer({storage:multer.memoryStorage()})`
39. MIME TYPE req.file.mimetype comes from request metadata and should NOT be blindly trusted as the real file type. if (!ALLOWED_TYPES[file.mimetype]) throw new ValidationError();
40. MAGIC BYTES Inspect actual file bytes to verify content matches declared type; prevents renamed malicious files. `const detected = await fileTypeFromBuffer(req.file.buffer);`
41. SAFE FILENAME Never trust original filename; generate a random server-side name. `const filename = `${crypto.randomUUID()}.${detected.ext}`;`
42. FILE SIZE Validate size both at multer boundary and handler/security layer. if (file.size > 510241024) throw new ValidationError('Too large');
43. S3 STORAGE Production file storage should use object storage rather than local container disk; instances then share the same durable files. `multer.memoryStorage() → s3.putObject(...) → save URL in DB`
44. LOCAL DISK PROBLEM Container restart can delete files; multiple instances don't share local filesystem and there's no built-in CDN/durability. Instance A uploads → Instance B may not see file

45. S3 PATTERN Upload binary to S3, store only object URL/key in DB. `await s3.putObject({Bucket,Key,Body:req.file.buffer});`
46. DAY-5 ARCHITECTURE TRIGGER HTTP concern? Controller. Business decision? Service. Database query? Repository. Cross-request state? Redis. File persistence? S3. Logs? Pino. Security headers? Helmet.

===== DAY 6 — TESTING + DOCKER + CI

1. UNIT TEST Tests ONE function/class in isolation; dependencies mocked; fast and deterministic.
`expect(calculateTotal(10,2)).toBe(20);`
2. INTEGRATION TEST Exercises multiple real layers, typically HTTP → Express → real test DB; catches boundary/integration bugs. `await request(app).post('/users').send({email:'a@test.com'}).expect(201);`
3. UNIT vs INTEGRATION Unit asks "does this logic work?"; integration asks "do these components work together?" Unit → mock repo; Integration → real test DB/HTTP.
4. JEST Jest is the test runner/assertion/mocking framework used for backend unit/integration tests.
`describe('UserService', () => { it('...', async () => {}) });`
5. ts-jest Allows Jest to execute TypeScript tests through TypeScript transformation. `preset: 'ts-jest', testEnvironment: 'node'`
6. TEST MATCH Jest discovers common test naming patterns such as `.test.ts` and `.spec.ts`. `testMatch: ['**/*.test.ts','**/*.spec.ts']`
7. AAA Arrange = setup; Act = execute; Assert = verify. `const input = ...; // Arrange const result = fn(input); // Act expect(result).toBe(...); // Assert`
8. `jest.fn()` Creates a fresh mock function whose return value/implementation/calls can be controlled.
`const repo = { find: jest.fn() }; repo.find.mockResolvedValue(user);`
9. `jest.spyOn()` Wraps an existing object's method so calls can be observed and/or implementation overridden. `const spy = jest.spyOn(bcrypt,'compare').mockResolvedValue(true);`
10. `jest.mock()` Mocks an entire imported module. `jest.mock('../repositories/user.repository');`
11. MOCK RESOLVED/REJECTED Use `mockResolvedValue` for async success and `mockRejectedValue` to simulate failures. `mockFn.mockResolvedValue(user); mockFn.mockRejectedValue(new Error('DB down'));`
12. MOCK ONCE Different behavior on successive calls can be simulated with `mockResolvedValueOnce`. `fn.mockResolvedValueOnce(null).mockResolvedValueOnce(user);`
13. MOCK ASSERTIONS Check call count, arguments and last call to verify interactions.
`expect(repo.find).toHaveBeenCalledTimes(1); expect(repo.find).toHaveBeenCalledWith('user-1');`
14. BEFOREALL Runs once before all tests in a suite; good for expensive shared setup. `beforeAll(async () => await setupDatabase());`
15. AFTERALL Runs once after suite; close DB/server/resources here. `afterAll(async () => await prisma.$disconnect());`

16. BEFORE EACH Runs before each test to reset/setup isolated state. `beforeEach(() => jest.clearAllMocks());`
17. AFTER EACH Runs after each test for cleanup when necessary. `afterEach(() => cleanup());`
18. CLEAR/RESET/RESTORE MOCKS `clearMocks` clears call history; `resetMocks` also resets implementations; `restoreMocks` restores spy originals. `clearMocks:true`, `restoreMocks:true`
19. SUPertest Supertest sends HTTP requests directly to Express app without needing a real listening port. `await request(app).get('/health').expect(200);`
20. INTEGRATION FLOW Supertest → Express middleware → controller → service → repository → test DB. `await request(app).post('/users').send(body).expect(201);`
21. TEST DATABASE Integration tests should use isolated test DB/data and cleanup between suites/tests to avoid flaky interactions. `beforeAll(Setup); afterAll(teardown);`
22. RUNINBAND Integration tests sharing one DB can interfere when Jest runs them in parallel; run serially. `jest --testPathPattern=integration --runInBand`
23. COVERAGE Coverage measures statements, branches, functions and lines executed by tests. `jest -coverage`
24. COVERAGE TARGET 80% is a useful practical target from the notes; 100% isn't automatically valuable if tests only cover trivial code. `coverageThreshold:{global:{branches:80,functions:80,lines:80,statements:80}}`
25. DOCKER Docker packages application + runtime/dependencies into an isolated image that runs consistently across environments. `docker build -t myapp .`
26. IMAGE vs CONTAINER Image = immutable blueprint; Container = running instance of an image. `docker run myapp`
27. DOCKERFILE Dockerfile describes how to build the application image. `FROM node:20-alpine WORKDIR /app`
28. WORKDIR Sets working directory for subsequent Docker commands. `WORKDIR /app`
29. COPY Copies files from build context into image. `COPY package*.json ./ COPY src ./src`
30. RUN Executes commands while BUILDING image. `RUN npm ci RUN npm run build`
31. CMD Default command executed when container starts; prefer `exec`/JSON form for signal handling. `CMD ["node","dist/index.js"]`
32. ENTRYPOINT vs CMD ENTRYPOINT defines fixed executable; CMD supplies default command/arguments that can be overridden. `ENTRYPOINT ["node"] CMD ["dist/index.js"]`
33. EXPOSE Documents the container port; does NOT publish it to host by itself. `EXPOSE 3000` `docker run -p 3000:3000 myapp`
34. ENV Sets environment variables inside image/container; secrets should NOT be baked into image. `ENV NODE_ENV=production`

35. DOCKER PID 1 Exec-form CMD makes Node the container's main process so it receives SIGTERM directly and can gracefully shut down. CMD ["node","dist/index.js"]
36. MULTI-STAGE BUILD Builder stage installs dev deps + compiles TS; production stage contains only compiled JS + production dependencies. FROM node:20-alpine AS builder RUN npm ci && npx tsc FROM node:20-alpine AS production
37. WHY MULTI-STAGE Keeps TypeScript/compiler/devDependencies/source out of production image → smaller image + smaller attack surface. builder → dist/ → production image
38. NON-ROOT USER Don't run production containers as root; create/use an unprivileged user. RUN adduser --system appuser USER appuser
39. HEALTHCHECK Docker can periodically test whether the service is healthy. HEALTHCHECK CMD wget -qO- http://localhost:3000/health || exit 1
40. .dockerignore Prevents node_modules, .git, .env, tests and other unnecessary/sensitive files entering Docker build context. node_modules/ .git/ .env **tests/**
41. DOCKER COMPOSE Compose defines multiple services and networking, e.g. Node app + Postgres + Redis. services: app: postgres: redis:
42. COMPOSE SERVICE DNS Containers on the same Compose network reach each other by service name, NOT localhost. DATABASE_URL=postgresql://postgres:5432/db REDIS_URL=redis://redis:6379
43. PORT MAPPING host:container maps host port to container port. ports:
 - "3000:3000"
44. depends_on Controls startup dependency; service_healthy waits for dependency health check rather than merely container startup. depends_on: postgres: condition: service_healthy
45. DOCKER NETWORK Compose creates a shared network so services can communicate by service name. app → postgres:5432 app → redis:6379
46. npm install vs npm ci npm install can update lockfile/version resolution; npm ci installs exactly from package-lock and is preferred in CI/Docker. npm ci
47. GITHUB ACTIONS CI workflow automatically runs checks on push/PR: checkout → Node setup → npm ci → lint → test → build. uses: actions/setup-node@v4
48. CI SEQUENCE A healthy backend pipeline should fail early on lint/tests before deployment/build artifacts are promoted. npm ci → npm run lint → npm test → npm run build
49. PARALLEL CI JOBS Independent jobs such as lint and test can run in parallel to reduce pipeline time. jobs: lint: ... test: ...
50. CI SECRETS Credentials belong in GitHub Actions Secrets/environment configuration, never committed into repository. \${ secrets.DATABASE_URL }
51. CI NODE MATRIX Matrix builds test the same code against multiple Node versions. strategy: matrix: node-version: [18,20,22]

52. HEALTH ENDPOINT Health endpoint exposes service/process + dependency state for Docker/load balancer probes. GET /health → {status:'healthy',checks:{database:'up',redis:'up'}}
53. LIVENESS Liveness answers "is the process alive?" and should be lightweight without dependency checks. GET /health/live → {status:'alive'}
54. READINESS/HEALTH Readiness/health answers "can this service actually serve traffic?" and may check DB/Redis. DB down → unhealthy/503
55. GRACEFUL DOCKER SHUTDOWN SIGTERM → stop accepting traffic → finish requests → close DB/Redis → exit. process.on('SIGTERM', gracefulShutdown);
56. DAY-6 MASTER FLOW Code → unit tests → integration tests → Docker build → healthcheck → CI lint/test/build → deploy. PR → GitHub Actions → npm ci → lint/test → build → Docker image → deployment.

=====

===== THE 30-SECOND

NODE.JS MASTER MAP

REQUEST ↓ Node http.Server ↓ Express middleware ↓ Route ↓ Controller ↓ Service ↓ Repository ↓ Prisma ↓ PostgreSQL

WHILE WAITING: Node main thread ↓ Event Loop ↓ libuv / OS async I/O ↓ callback/promise ↓ Event Loop

CPU HEAVY? ↓ Worker Thread

NEED MULTIPLE HTTP PROCESSES? ↓ Cluster / PM2 / containers

NEED SHARED STATE? ↓ Redis

NEED AUTH? ↓ bcrypt password ↓ JWT access token ↓ refresh token + rotation

NEED VALIDATION? ↓ Zod at API boundary

NEED DATABASE SAFETY? ↓ Constraints + indexes + transactions + connection pool

NEED PRODUCTION STRUCTURE? ↓ Route → Controller → Service → Repository

NEED PRODUCTION LOGGING? ↓ Pino + correlation/request ID

NEED SECURITY? ↓ Helmet + CORS + rate limit + auth/authz + input validation ↓ XSS / CSRF / IDOR / SQL injection protection

NEED FILE UPLOAD? ↓ Multer → validate size + MIME + magic bytes ↓ S3 ↓ URL/key in PostgreSQL

NEED CONFIDENCE? ↓ Unit tests + Integration tests

NEED DEPLOYMENT? ↓ Docker multi-stage ↓ Compose / Kubernetes / platform ↓ GitHub Actions CI

=====

===== MOST IMPORTANT "IF I SEE THIS, THINK THAT" TRIGGERS

V8 → executes JS + JIT + heap/stack libuv → async I/O + event loop + thread pool Event Loop → schedules callbacks without blocking on I/O nextTick → highest-priority microtask Promise.then → Promise microtask setImmediate → check phase setTimeout → timers phase

fs/crypto/zlib → may use libuv thread pool HTTP/TCP → OS async I/O

CPU-heavy JS → Worker Thread HTTP scaling → Cluster shared memory → SharedArrayBuffer separate memory → Cluster

stream → chunks + low memory pipe() → pipeline + backpressure write=false → producer must slow down drain → resume producer

require → CommonJS import → ESM module.exports → CJS export

req.params → URL path req.query → filters/options req.body → payload req.headers → metadata/auth req.user → authenticated identity

401 → authentication problem 403 → authorization problem 404 → resource/route missing 409 → conflict 429 → rate limit 500 → server failure

next() → continue middleware next(err) → jump to error handler 4 args → Express error middleware

Promise.all → all must succeed allSettled → need every result race → first settled any → first successful

bcrypt → passwords JWT → signed identity/claims HS256 → shared secret RS256 → private/public key JWKS → distribute public keys access token → short-lived/stateless refresh token → long-lived/session renewal rotation → replace refresh token every use

Zod → runtime input validation + TS inference interface → compile-time only discriminated union → safe variant narrowing

JOIN → combine related tables index → faster lookup transaction → atomic multiple writes N+1 → too many repeated DB queries pool → reuse DB connections include → load relations select → control returned fields cursor → scalable pagination

Route → path + middleware Controller → HTTP translation Service → business rules Repository → DB DI → inject dependency SOLID → maintainability

Pino → structured logs correlation ID → trace one request across logs Helmet → security headers rate limit → abuse protection IDOR → verify resource ownership magic bytes → real file type Multer → multipart upload S3 → durable shared file storage

jest.fn → new mock spyOn → observe existing method jest.mock → mock module AAA → Arrange → Act → Assert Supertest → HTTP integration testing runInBand → serial integration tests coverage → statements/branches/functions/lines

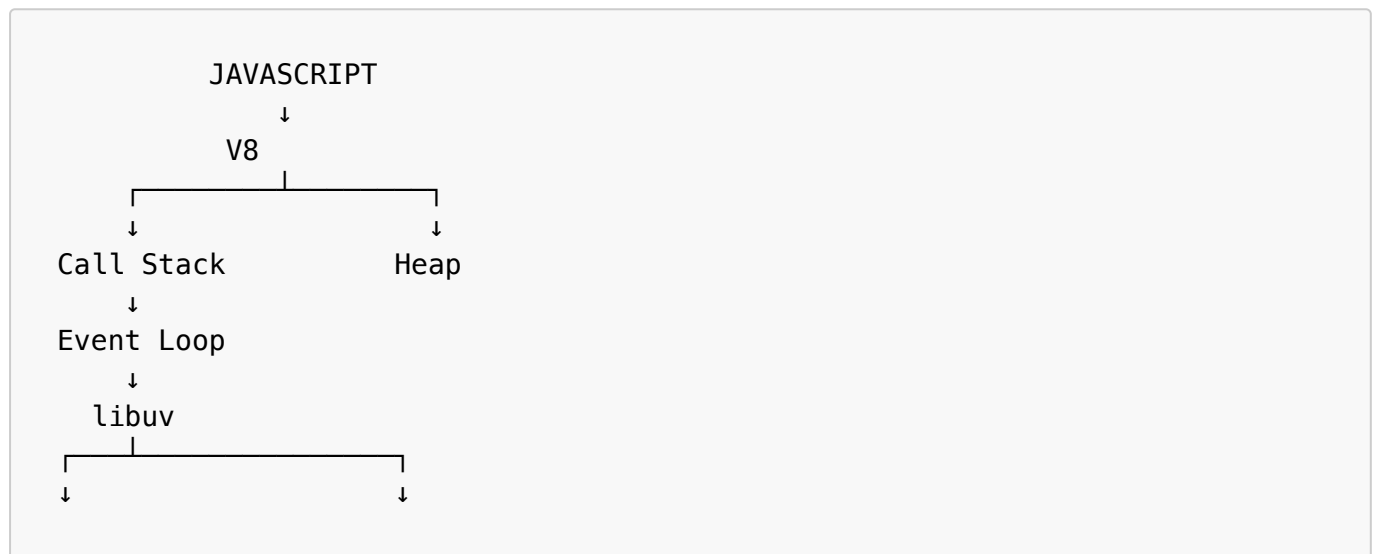
Docker image → application blueprint container → running image Dockerfile → build instructions CMD → startup command multi-stage → small production image .dockerignore → smaller + safer build context
Compose → multi-container local environment healthcheck → container health npm ci → reproducible CI
install GitHub Actions → automated CI/CD

=====

===== FINAL MENTAL MODEL

Node.js is NOT "a single thread doing everything."

Think:



OS async I/O Thread Pool ↓ fs / crypto / zlib

CPU-heavy JS ↓ Worker Thread

Many HTTP processes ↓ Cluster / Containers

Application architecture ↓ Route ↓ Controller ↓ Service ↓ Repository ↓ PostgreSQL

Shared state ↓ Redis

Authentication ↓ bcrypt → JWT access → refresh token

Production ↓ Validation + Security + Logging ↓ Tests ↓ Docker ↓ CI/CD